Indian Institute of Information Technology, Bhopal

(Department -CSE)



PROJECT REPORT

Subject Name: Advance Programming Lab - I (Python)

Team Members:

1. Vikrant Krishna 25U024010

2. Animesh Gupta 25U024044

Project Title : Text Compression and Decompression Using Huffman Coding

SUBMITTED TO : Dr. Gagan Vishwakrma

Abstract:

HuffX is a lossless data compression and decompression system based on Huffman Coding, a greedy algorithm used to generate variable-length prefix codes according to symbol frequencies. The primary objective of the project is to reduce the storage requirements of text and other supported input data while preserving the original information exactly.

The system uses Python to implement frequency analysis, min-heap-based priority queue operations, Huffman tree construction, code generation, encoding, and decoding. A web-based interface provides users with a convenient way to interact with the compression engine. The project also explores configurable symbol widths to study how the grouping of input data affects compression performance.

The system is evaluated using different input types, including repetitive text and source code. Compression efficiency is measured using file sizes, space savings, and successful reconstruction of the original input. The project demonstrates the practical application of greedy algorithms, binary trees, priority queues, and file processing in software development.

Problem statement:

Text files can contain repeated characters, words, and patterns. Storing every character using a fixed-width representation may use more bits than necessary for data with uneven symbol frequencies. Huffman coding assigns shorter codes to frequent symbols and longer codes to less frequent symbols, reducing the number of bits required for suitable input data.

Our project aims to implement this technique in a usable system that compresses data and reconstructs it without information loss.

Objectives:

- ➔ Implement Huffman coding using Python.
- ➔ Use a min-heap to construct a Huffman tree efficiently.
- ➔ Generate prefix-free binary codes from symbol frequencies.
- ➔ Support compression and exact decompression.
- ➔ Create a usable interface for interacting with the compression engine.
- ➔ Measure compression performance across different types of input.
- ➔ Explore how different symbol widths affect compression efficiency.
- ➔ Validate that decompressed output matches the original input**.**

WORK Flow:

STEP 1

Read input data

STEP 2

Count symbol frequencies

STEP 3

Insert symbols into a min-heap

STEP 4

Merge the two lowest-frequency nodes repeatedly

STEP 5

Generate prefix-free Huffman codes

STEP 6

Encode and pack the bits

STEP 7

Decode and verify the original data

Technology Stack and Libraries

HuffX is developed using Python and Next.js, combining an algorithmic compression engine with a web-based user interface.

Frontend Technologies

- Next.js: Used to develop the web application and organize the frontend.
- React: Used to create reusable user-interface components.
- JavaScript and JSX: Used to implement frontend interactions and application logic.

Backend and Algorithm Technologies

- Python: Implements the Huffman coding algorithm, including frequency analysis, tree construction, encoding, decoding, and file processing.
- heapq: Provides min-heap operations for efficiently constructing the Huffman tree.
- collections.Counter: Calculates the frequency of symbols in the input data.
- Python file I/O: Reads input files and writes compressed and decompressed outputs**.**

Development Tools

- Visual Studio Code: Used for development and debugging.
- Git: Used for version control.
- GitHub: Used to host the project repository and manage source code.

The project applies fundamental data structures and algorithms, including binary trees, priority queues, and greedy algorithm techniques. The selected technologies support the implementation of lossless compression, file processing, and an accessible web interface.

Multi-symbol-width feature:

This deserves its own subsection because it is a distinctive feature of your implementation.

Symbol width	Chunk size
8-bit	1 byte
32-bit	4 bytes
64-bit	8 bytes
128-bit	16 bytes

Complexity analysis:

Operation	Time complexity
Frequency counting	(O(N))
Heap construction	(O(K))
Huffman tree construction	(O(Klog K))
Code generation	(O(K))
Encoding and decoding	Typically (O(N)) in the number of processed symbols/bits, under the implementation's lookup assumptions

Conclusion, limitations and future scope:

Limitations to discuss honestly:

- Compression is not guaranteed to reduce every file's size.
- Codebook and header overhead can be significant for small inputs.
- Compression efficiency depends on the input data and symbol-width selection.
- Huffman coding alone may be less effective than more advanced compression pipelines for certain file types.

Future scope:

- Add support for additional file formats.
- Improve the automatic selection of symbol width.
- Compare Huffman coding with other lossless compression algorithms.
- Add more extensive benchmarking and visualizations.
- Improve error handling and validation for corrupted compressed files.